# TdP — I contenitori di dati in Python

Collezioni, liste, array, mappe, set, code e code prioritarie: come funzionano e quando usarle

## Introduzione: perché esistono così tanti contenitori

Python mette a disposizione diverse strutture dati che permettono di creare dei **contenitori di oggetti**: liste, tuple, set e dizionari sono le più comuni, ma esistono anche varianti più specializzate (deque, code prioritarie, namedtuple, Counter…). Non sono strutture equivalenti tra loro: ciascuna è stata pensata per uno scopo preciso, e la scelta giusta dipende da **come** il codice dovrà accedere ai dati, non solo da cosa deve contenere.

Le domande da porsi, prima di scegliere un contenitore, sono sostanzialmente tre:

- Ho bisogno di recuperare un elemento tramite una **chiave** (posizione casuale, senza scorrere tutto), oppure mi basta scorrere la collezione in ordine?
- Mi interessa mantenere un **ordinamento** degli elementi, oppure l'ordine non ha importanza (e magari non voglio nemmeno duplicati)?
- Gli elementi devono poter essere **modificati** dopo la creazione del contenitore, oppure la collezione deve restare immutabile?

Le slide del corso riassumono queste famiglie in uno schema:

| Famiglia | Tipi | Caratteristica |
|---|---|---|
| **Dictionaries, Maps, Hash Tables** | `dict`, `OrderedDict`, `defaultdict`, `ChainMap`, `MappingProxyType` | Accesso per chiave, non ordinati |
| **Array Data Structures** | `list`, `tuple`, `array`, `str` | Sequenze ordinate |
| **Records, Structs, Data Transfer Objects** | `dict`, `tuple`, `class`, `dataclass`, `namedtuple`, `NamedTuple`, `struct` | Campi eterogenei con nome |
| **Sets, Multisets** | `set`, `frozenset`, `Counter` | Elementi distinti, non ordinati |
| **Stacks (LIFO)** | `list`, `deque`, `LifoQueue` | Ultimo entrato, primo uscito |
| **Queues (FIFO)** | `list`, `deque`, `Queue` | Primo entrato, primo uscito |
| **Priority Queues** | `list`, `heapq`, `PriorityQueue` | Estrazione per priorità |

In generale, come indicato anche nelle slide: alcuni tipi sono estremamente versatili (`list`, `dict`) e vanno bene quasi sempre come prima scelta; altri sono delle "versioni migliorate"

di tipi di base (ad esempio deque rispetto a `list`, oppure `defaultdict` rispetto a `dict`) e si scelgono quando si conosce già un limite specifico del tipo base; altri ancora sono molto specializzati e si usano solo in contesti particolari (ad esempio nel calcolo parallelo).

---

## Ordinare una collezione: `__eq__` e `__lt__`

Prima di entrare nel dettaglio dei singoli contenitori vale la pena capire **come Python decide se due oggetti sono uguali o se uno è minore dell'altro**, perché da questo dipende sia il comportamento dei set e dei dizionari (che hanno bisogno di sapere se due chiavi sono uguali), sia la possibilità di ordinare una collezione.

L'ordinamento e il confronto tra oggetti di una collezione si basano su due *dunder method* che possono essere definiti nella classe:

- **`__eq__`** definisce se due oggetti sono considerati uguali o no;
- **`__lt__`** restituisce vero/falso a seconda che un oggetto sia considerato minore di un altro.

Alcune collezioni built-in implementano già questi metodi: le stringhe, le liste e le tuple confrontano i propri elementi in ordine da sinistra a destra; i dizionari supportano `__eq__` ma non `__lt__`; i set supportano sia `__eq__` sia `__lt__`, ma in questo caso `__lt__` ha un significato diverso: indica che un set è (propriamente) **sottoinsieme** di un altro.

Per le classi definite da noi, la `@dataclass` genera automaticamente un `__eq__` che confronta tutte le proprietà della classe (due istanze sono uguali se tutti i campi coincidono), ma **non** genera un `__lt__`: se serve, va scritto a mano, oppure si può passare `order=True` al decoratore (`@dataclass(order=True)`) per delegare l'ordinamento al confronto, campo per campo, degli attributi. È anche possibile passare `eq=False` per non generare `__eq__`, e si può escludere un singolo campo dal confronto/ordinamento inizializzandolo con `field(compare=False)`.

Quando si vuole ordinare una collezione secondo un criterio specifico, esistono diversi modi per specificare la chiave di ordinamento (parametro key di `sort`/`sorted`):

```
key=operator.itemgetter('keyname')   # sort by dictionary['keyname']
key=operator.itemgetter(itemnumber)  # sort by list/tuple[itemnumber]
key=operator.attrgetter('attrname')  # sort by object.attrname
key = lambda obj: something(obj)     # criterio arbitrario, il più generale
                           # es. list.sort(key=lambda x: x.proprietà)
```

Il metodo con `attrgetter` (modulo `operator`) è comodo quando si vuole ordinare per una proprietà precisa dell'oggetto; la `lambda` è invece il metodo più generale, perché permette di scrivere qualunque criterio di ordinamento, anche complesso.

---

## 1. Dizionari (`dict`) — accesso per chiave

Un **dizionario** è una struttura dati che mappa due elementi, chiave e valore, con sintassi `{ key: val }`. Ha senso usarlo quando la collezione serve per salvare dei dati e poi si deve

andare a riprendere un particolare elemento in posizione casuale, senza voler aspettare il tempo necessario per scorrere tutta una lista.

Perché una chiave possa essere usata in un dizionario deve essere **hashable**, cioè dotata di una funzione di hash: una funzione di *mapping* che associa a un elemento un valore intero univoco rispetto all'argomento in ingresso. Un oggetto è hashable se è possibile calcolarne un valore di hash immutabile nel tempo, e se è comparabile (cioè definisce __eq__): due istanze di una classe sono considerate uguali se, e solo se, hanno lo stesso hash (a == b ⇒ hash(a) == hash(b)). Per garantire che un oggetto abbia una funzione di hash si definisce il metodo __hash__, che restituisce un intero univoco per quell'istanza; spesso conviene delegare l'hash alle variabili usate anche nel confronto, calcolando l'hash di una tupla contenente tutte le proprietà rilevanti dell'oggetto.

La funzione di hash mappa ogni oggetto in un intero a 64 bit. Se si riesce a mappare ogni istanza del programma in hash diversi, diventa possibile una ricerca molto efficiente, perché il valore di hash può essere usato direttamente come "indirizzo" nella struttura interna del dizionario: per questo i dizionari sono efficienti nell'inserire, cercare e rimuovere valori, indipendentemente dall'ordinamento.

**Metodi principali:**

```
d[key] = v          # scrive/aggiorna il valore associato a key
v = d[key]           # legge -- restituisce KeyError se key non esiste
v = d.get(key, default)  # legge senza KeyError; se assente rende il default
d.pop(key)           # restituisce il valore e lo rimuove dal dizionario
d.clear()            # elimina tutte le coppie chiave-valore
d.keys()             # restituisce tutte le chiavi definite nel dizionario
d.values()           # restituisce tutti i valori salvati nel dizionario
d.items()            # restituisce le coppie (chiave, valore)
key in d              # verifica se key è presente nel dizionario
```

Esempio pratico tratto dagli appunti (catalogo prodotti indicizzato per codice):

```
catalogo = {"LAP001": pr("laptop", 1200.0),
            "LAP002": pr("laptop pro", 2300.0),
            "MAU002": pr("mouse", 20.0), "AUR001": pr("auricolari", 250.0)}
prod = catalogo["LAP001"]
p4 = catalogo.get("NonEsiste")                      # restituisce None
p5 = catalogo.get("NonEsiste2", pr("sconosciuto", 0.0))  # restituisce default

for k in catalogo.keys(): print(k)
for v in catalogo.values(): print(v)
for k, v in catalogo.items():
    print(f"cod {k} è associata a {v}")

rimosso = catalogo.pop("LAP001")

# dict comprehension: costruisce un nuovo dizionario in modo compatto
prezzi = {codice: prod.prezzo_unitario for codice, prod in catalogo.items()}
```

### Varianti del dizionario standard

- **`collections.defaultdict(tipo)`**: se si prova a leggere una chiave che non esiste, invece di sollevare `KeyError` la chiave e il relativo valore vengono creati automaticamente, usando come valore di default un'istanza del tipo passato come argomento (ad esempio `defaultdict(int)` crea automaticamente `0` per una chiave mancante). Va importato dal modulo `collections`.
- **`types.MappingProxyType(normal_d)`**: crea una "vista" di sola lettura di un dizionario già esistente. Non è una copia indipendente: maschera tutti i metodi che modificherebbero il dizionario originale, sollevando `TypeError: 'mappingproxy' object does not support item assignment` se si tenta di scrivere.

**Quando usarlo:** ogni volta che serve indicizzare/recuperare oggetti tramite una chiave (codice prodotto, id cliente, nome...) senza voler scorrere l'intera collezione; `defaultdict` quando si accumulano valori per chiavi non ancora note a priori (contatori, raggruppamenti); `MappingProxyType` quando si vuole esporre un dizionario "di sola lettura" verso l'esterno.

---

## 2. Array: `list`, `tuple`, `str` e array specializzati

Tutte le strutture che **prevedono un ordinamento** (cioè una posizione precisa per ogni elemento) vengono chiamate genericamente *array*, e comprendono liste, tuple e stringhe.

### `list`

La `list` rappresenta una sequenza ordinata di oggetti, **versatile e mutabile**: si possono aggiungere, rimuovere e modificare elementi liberamente. È probabilmente il contenitore più usato in assoluto, la scelta di default quando serve semplicemente una sequenza di oggetti su cui iterare, a cui aggiungere elementi o da ordinare.

```
carrello = [p1, p2, p3, pr("tablet", 700.0)]

# aggiungere elementi
carrello.append(elem)          # aggiunge in coda un elemento (come ultimo)
carrello.insert(x, elem)       # aggiunge un elemento nella posizione x
carrello.extend([lista_elem])  # aggiunge una lista di elementi

# rimuovere elementi
carrello.pop()          # rimuove e restituisce l'ultimo elemento
carrello.pop(2)         # rimuove l'elemento in posizione 2
carrello.remove(p1)     # rimuove la prima occorrenza di p1
carrello.clear()        # svuota la lista

# ordinamento (questi metodi modificano la lista originale)
carrello.sort()  # ordinamento naturale (richiede __lt__)
carrello.sort(reverse=True)                  # ordine inverso
carrello.sort(key=lambda x: x.prezzo_unitario, reverse=True)
carrello_ordinata = sorted(carrello)         # crea una lista NUOVA, ordinata
```

```
carrello.reverse()                              # inverte l'ordine della lista

# copie
carrello_copy = carrello.copy()  # shallow copy: stessi oggetti, lista diversa
carrello_copy2 = copy.deepcopy(carrello)
# deep copy: copia anche il contenuto degli elementi
```

Da notare la differenza cruciale tra **shallow copy** e **deep copy**: `.copy()` crea una nuova lista che contiene però *gli stessi oggetti* della lista originale (se modifico un oggetto contenuto, la modifica è visibile da entrambe le liste); `copy.deepcopy()` copia ricorsivamente anche il contenuto di ogni elemento, ottenendo una struttura completamente indipendente.

### `tuple`

La `tuple` è la versione **immutabile** della lista: una volta creata, gli elementi non possono essere aggiunti né rimossi (anche se, se contiene oggetti mutabili, questi possono comunque essere modificati al loro interno). Proprio perché immutabile, una tupla è hashable se lo sono tutti i suoi elementi — il suo hash si ottiene combinando gli hash dei singoli elementi — e può quindi essere usata, ad esempio, come chiave di un dizionario, cosa impossibile per una lista.

```
sede_principale = (45, 8)   # latitudine e longitudine
print(f"lat {sede_principale[0]}, long {sede_principale[1]}")

# tupla di tuple
AliquoteIVA = (("standard", 0.22), ("ridotta", 0.10),
               ("alimentari", 0.04), ("esente", 0.0))
for i, valore in AliquoteIVA:
    print(f"{i}: {valore*100}%")

# restituire più valori da una funzione tramite una tupla
def calcola_statistiche_carrello(carrello):
    prezzi = [p.prezzo_unitario for p in carrello]
    return (sum(prezzi), sum(prezzi)/len(prezzi), max(prezzi), min(prezzi))

tot, media, mx, mn = calcola_statistiche_carrello(carrello)  # unpacking
# tot, *altri_campi = calcola_statistiche_carrello(carrello)
# -> solo il 1° elemento finisce in tot
```

**Quando usarla:** tutte le volte in cui un insieme di valori è logicamente "fisso" (coordinate, una riga di dati che non deve cambiare, il valore di ritorno multiplo di una funzione) e, in particolare, ogni volta che quei valori devono poter essere usati come chiave di un dizionario o elemento di un set.

### `str`

Una stringa è una sequenza di caratteri **immutabili**; condivide con liste e tuple il comportamento di confronto elemento per elemento da sinistra a destra.

### Array specializzati

- **`array.array`**: concettualmente simile a una lista, ma con il vincolo che tutti gli elementi devono essere dello stesso tipo, dichiarato al momento della creazione: si specifica il tipo di dato che conterrà e poi si passano gli elementi uno dopo l'altro. Utile quando si vuole un contenitore più compatto ed efficiente in memoria rispetto a una lista generica.
- **`bytes`** e **`bytearray`**: array di byte, rispettivamente nella versione immutabile (`bytes`) e mutabile (`bytearray`).

---

## 3. Record: contenitori di campi eterogenei

I **record** sono collezioni che contengono istanze di elementi di tipo eterogeneo — ad esempio una classe, una dataclass, un dizionario o una tupla — dove ogni campo ha un significato proprio (a differenza di una lista, in cui tutti gli elementi hanno tipicamente lo stesso ruolo).

- **`collections.namedtuple()`**: crea una tupla in cui ad ogni posizione viene associato anche un nome, permettendo di accedere agli elementi sia per indice sia per nome. Viene creata come classe del modulo `collections`.
- **`typing.NamedTuple`**: sintassi simile a quella di una `@dataclass`, crea una classe con dei parametri tipizzati che verranno poi valorizzati.

**Quando usarli:** quando si vuole rappresentare un piccolo record di dati eterogenei con la leggerezza e l'immutabilità di una tupla, ma senza perdere la leggibilità che dà l'accesso per nome ai singoli campi (in alternativa a una classe vera e propria o a una `@dataclass`, quando non servono metodi ulteriori).

---

## 4. `set`, `frozenset` e `Counter`

### `set`

Un **set** è un contenitore **mutabile** di oggetti hashable in cui **i duplicati non sono permessi**: rappresenta insiemisticamente una collezione di elementi distinti, con sintassi `nome = {elementi}`. Ha senso usarlo quando interessa solo *se* un elemento è presente, non quante volte o in che posizione, oppure quando servono le classiche operazioni insiemistiche.

```
categorie = {"Gold", "Silver", "Bronze", "Gold"}
# il duplicato "Gold" viene eliminato
categorie2 = {"Platino", "Elite", "Gold"}

categorie_all = categorie | categorie2       # unione
categorie_comuni = categorie & categorie2    # intersezione
categorie_esclusive = categorie - categorie2 # differenza (solo in categorie)
categorie_simmetrica = categorie ^ categorie2 # differenza simmetrica
```

```
s = {pr("laptop", 1200.0), pr("mouse", 20.0), pr("tablet", 700.0)}
# richiede __hash__
s.add(pr("auricolari", 250.0))     # aggiunge un elemento
s.update([elem1, elem2])           # aggiunge più elementi
s.remove(elem)     # rimuove un elemento, KeyError se non esiste
s.discard(elem)  # rimuove un elemento, nessun errore se non esiste
s.pop()            # rimuove e restituisce un elemento arbitrario
s.clear()                          # svuota il set

# metodi equivalenti agli operatori
s.union(s1); s.intersection(s1); s.difference(s1); s.symmetric_difference(s1)
s1.issubset(s)     # True se tutti gli elementi di s1 sono contenuti in s
s1.issuperset(s)   # True se tutti gli elementi di s sono contenuti in s1
s1.isdisjoint(s)   # True se s e s1 non hanno elementi in comune
```

Esiste anche il **frozenset**, la versione immutabile del set: una volta creato non può più essere modificato, e proprio per questo — essendo immutabile — può essere usato come chiave di un dizionario o come elemento di un altro set, cosa non permessa a un set normale.

### Counter

Il `Counter` (`collections.Counter`) è un oggetto specializzato che si comporta come un dizionario in cui le chiavi sono gli elementi distinti passati come argomento e i valori indicano quante volte ciascun elemento è comparso. È lo strumento giusto ogni volta che serve **contare le occorrenze** di qualcosa.

```
categorie = [c.categoria for c in lista_clienti]
categorie_counter = Counter(categorie)
print(f"più frequenti: {categorie_counter.most_common(2)}")  # i 2 più numerosi
print(f"totale: {categorie_counter.total()}")

vendite_gennaio = Counter({"Laptop": 13, "Tablet": 15})
vendite_febbraio = Counter({"Laptop": 3, "Stampante": 1})
vendite_bimestre = vendite_gennaio + vendite_febbraio
# somma i conteggi per chiave
differenza = vendite_gennaio - vendite_febbraio
# sottrae i conteggi per chiave
vendite_gennaio["Laptop"] += 4  # modifica un conteggio al volo
```

Altri metodi utili: `c.most_common(n)` restituisce gli n elementi più frequenti (senza argomenti restituisce tutti gli elementi ordinati per frequenza); `c.total()` restituisce la somma di tutti i conteggi.

**Quando usarli:** `set`/`frozenset` quando conta solo l'appartenenza e servono operazioni insiemistiche, con la garanzia di unicità degli elementi; `Counter` ogni volta che serve costruire una distribuzione di frequenza a partire da una collezione di dati.

---

## 5 e 6. Pile (stack, LIFO) e code (queue, FIFO)

Pile e code sono concettualmente simili a una lista, ma con **modalità diverse di accesso** ai valori contenuti: nelle **pile (LIFO — Last In, First Out)** si assume che l'ultimo oggetto inserito sia il primo ad essere estratto; nelle **code (FIFO — First In, First Out)** si assume invece che il primo oggetto inserito sia anche il primo ad essere estratto.

È possibile rappresentarle con una semplice lista `s = list()`: si usa `s.append(new)` per aggiungere un nuovo elemento e, per estrarlo, `old = s.pop()` per togliere l'ultimo elemento (comportamento a pila) oppure `old = s.pop(0)` per togliere il primo elemento (comportamento a coda). Il problema è che `pop(0)` è un'operazione **lenta**: tutti gli elementi rimanenti devono essere spostati di una posizione, quindi la complessità è O(n), con n lunghezza della lista.

Per risolvere questo problema si usa il **deque** (*double-ended queue*, `collections.deque()`): una lista a cui è possibile aggiungere e togliere elementi sia a sinistra sia a destra, **senza differenza di complessità computazionale** in nessuno dei due casi. Ogni volta che ha senso gestire gli elementi "in ordine di arrivo", conviene quindi usare un deque piuttosto che una lista.

**Come coda FIFO:**

- append e `popleft` — la scelta più comune;
- `appendleft` e pop — anche possibile, stessa efficienza.

**Come pila LIFO:**

- append e pop — la scelta più comune (per questo caso, andrebbe bene anche una semplice lista);
- `appendleft` e `popleft` — anche possibile, stessa efficienza.

```
d = deque()                 # nuovo deque vuoto
d = deque(iterable)         # deque a partire da un iterabile
d = deque(maxlen=N)
# ospita al massimo N elementi, scartando i più vecchi
d.extend(iterable)          # aggiunge una lista di elementi in coda
d.extendleft(iterable)      # aggiunge una lista di elementi in testa
d.rotate(n)                 # ruota gli elementi di n posizioni
d[i]                        # accede a un elemento (più lento che nelle liste)
d.index(x), d.insert(i, x), d.remove(x), d.reverse()  # come nelle liste
```

Esempio pratico, una coda ordini FIFO:

```
coda_ordini = deque()
for i in range(1, 10):
    cliente = cr(f"Cliente {i}", f"cliente{i}@polito.it", "Gold")
    prodotto = pr(f"Prodotto{i}", 100.0*i)
    ordine = Ordine([RigaOrdine(prodotto, 1)], cliente)
    coda_ordini.append(ordine)

while coda_ordini:
    ordine_corrente = coda_ordini.popleft()
    print(f"Sto gestendo l'ordine del cliente: {ordine_corrente.cliente}")
```

**Quando usarli:** una `list` va benissimo per una pila semplice (append/pop dalla fine); appena serve invece rimuovere elementi anche dalla testa della collezione con frequenza (una coda FIFO, o un buffer a doppia estremità), conviene passare a deque per evitare il costo O(n) di `pop(0)`.

---

## 7. Code prioritarie (priority queue)

Le **code prioritarie** sono collezioni simili a una lista in cui l'inserimento avviene normalmente (si aggiunge a destra), ma l'estrazione (pop) restituisce sempre l'oggetto con **priorità più elevata**, indipendentemente dall'ordine di inserimento. Per far funzionare questo meccanismo, ad ogni oggetto aggiunto va associato anche un valore di priorità: si può ad esempio creare una tupla (`priority, value`) e poi ordinare in base alla priorità, oppure definire un metodo `__lt__` sull'oggetto che confronti le priorità.

Piuttosto che riordinare l'intera collezione ogni volta (operazione costosa), si usano strutture dedicate: il modulo **`heapq`** e la classe **`queue.PriorityQueue`** gestiscono le priorità internamente costruendo una gerarchia ad albero (heap) tra gli oggetti, che garantisce che il pop restituisca sempre, in modo efficiente, l'elemento con priorità più elevata, senza dover risortare l'intera lista ad ogni inserimento.

**Quando usarle:** ogni volta che gli elementi da processare non vanno gestiti in ordine di arrivo, ma secondo un criterio di priorità che può anche cambiare dinamicamente (ad esempio nella gestione di eventi, nella schedulazione di task, o — come visto anche nel capitolo sui cammini minimi — nell'implementazione efficiente dell'algoritmo di Dijkstra, dove il prossimo nodo da esplorare è sempre quello con distanza stimata minore).

---

## Schema riassuntivo delle operazioni sui contenitori

La tabella seguente (basata sullo schema sinottico delle slide) confronta, per i tipi principali, come si esprime la stessa operazione logica:

| Operazione | `str` | `list` | `tuple` | `set` | `dict` |
|---|---|---|---|---|---|
| Creare | `"abc"` | `[a, b, c]` | `(a, b, c)` | `{a, b, c}` | `{a:x, b:y}` |
| Creare vuoto | `""` | `[]` o `list()` | `()` o `tuple()` | `set()` | `{}` o `dict()` |
| Accedere all'i-esimo elemento | `s[i]` | `l[i]` | `u[i]` | — | `d[k]`, `d.get(k, default)` |
| Modificare l'i-esimo elemento | — | `l[i]=x` | — | — | `d[k]=x` |
| Aggiungere un elemento (modifica) | — | `l.append(x)` | — | `s.add(x)` | `d[k]=x` |

| Operazione | str | list | tuple | set | dict |
|---|---|---|---|---|---|
| Aggiungere in posizione (nuovo valore) | `s+'x'` | `l[x]` (insert) | `u+(x,)` | — | — |
| Unire due contenitori (modifica) | — | `l.extend(l1)` | — | `s.update(t1)` | — |
| Unire due contenitori (nuovo valore) | `s+s1` | `l+l1` | `u+u1` | `s.union(t1)`, `s\|t1` | — |
| Contiene un valore? | `x in s` | `x in l` | `x in u` | `x in s` | `k in d`; `x in d.values()` |
| Dove si trova un valore? | `s.find(x)`, `s.index(x)` | `l.index(x)` | `u.index(x)` | — | — |
| Rimuovere per indice | — | `l.pop(i)`, `l.pop()` | — | — | `d.pop(k)` |
| Rimuovere per valore | — | `l.remove(x)` | — | `s.remove(x)`, `s.discard(x)` | — |
| Ordinare (modifica) | — | `l.sort()` | — | — | — |
| Ordinare (nuovo valore) | `sorted(s)` | `sorted(l)` | `sorted(u)` | `sorted(s)` | `sorted(d)` (chiavi), `sorted(d.items())` |

---

## In sintesi: quale contenitore scegliere

- Serve **accedere per chiave** in modo rapido, senza scorrere tutto? → **`dict`** (`defaultdict` se le chiavi possono non esistere ancora, `MappingProxyType` per una vista di sola lettura).
- Serve una **sequenza ordinata e modificabile** di oggetti (caso più comune)? → **`list`**.
- Serve un insieme di valori **fisso** che, ad esempio, deve poter fare da chiave di un dizionario o elemento di un set? → **`tuple`** (o `namedtuple`/`NamedTuple` se conviene accedere ai campi per nome).
- Interessa solo **l'appartenenza** di un elemento, senza duplicati, con operazioni insiemistiche? → **`set`** (`frozenset` se deve essere immutabile/hashable).
- Serve **contare le occorrenze** di elementi in una collezione? → **`Counter`**.
- Serve gestire elementi **in ordine di arrivo** (LIFO o FIFO), con inserimenti/rimozioni frequenti da entrambe le estremità? → **`deque`** (una `list` semplice basta solo per una pila pura, append/pop dalla fine).
- Serve estrarre sempre l'elemento con **priorità più alta**, indipendentemente dall'ordine di arrivo? → **`heapq`** o **`queue.PriorityQueue`**.